StratIQ

StratIQ turns a business KPI anomaly into an evidence-backed investigation.

A traditional dashboard can tell a business leader that **revenue, sales, conversion, or loan disbursals fell**. The harder questions remain:

Is the change actually significant? Why did it happen? What evidence supports each explanation? And what should the business do next?

StratIQ is designed to answer those questions by combining statistical signal detection, structured business data, unstructured evidence, LLM-assisted hypothesis generation, deterministic confidence scoring, and decision-oriented recommendations.

1. The Business Problem

Business teams already have dashboards. The problem is what happens **after the dashboard raises an alarm**.

When a KPI suddenly moves, analysts typically need to investigate across multiple disconnected sources:

- sales and transaction systems
- operations and branch data
- pricing and policy changes
- customer support tickets
- field or dealer feedback
- news and external events

The result is a manual process of:

detect → cross-reference → form hypotheses → validate → decide

This creates three recurring problems.

Signal vs. Noise

Not every KPI movement represents a real business event.

A metric may move because of normal volatility, seasonality, or random variation. Investigating every fluctuation wastes analyst time and creates false alarms.

Correlation → Explanation → Action

A dashboard can show that two things moved together. It rarely determines which explanation is best supported or what a decision-maker should investigate next.

Ambiguity

Several causes can exist simultaneously.

A system that always produces a single confident answer risks turning incomplete evidence into false certainty.

2. What StratIQ Does

StratIQ introduces an investigation layer between **business intelligence and business action**.

The core workflow is:

KPI movement

↓

Signal detection

↓

Evidence fusion

↓

Competing hypotheses

↓

Confidence scoring

↓

Recommendation

OR

Evidence gap + next action

The principle is simple:

Do not just describe the anomaly. Investigate it.

3. Prototype Scenario : Suvidha Finserv

For the working prototype, we use an illustrative **Suvidha Finserv** two-wheeler lending scenario.

The business problem:

Two-wheeler loan disbursals in the Pune cluster fall by approximately 18% over the detected anomaly period.

The system must determine:

1. Is this a genuine anomaly?
2. Is the movement local or part of a broader trend?
3. What evidence exists around the anomaly?
4. Which competing explanations are best supported?
5. How confident should the business be?
6. What should be investigated next?

The scenario contains five plausible business drivers:

- tighter internal credit/CIBIL underwriting
- field-agent attrition
- monsoon/flood-related dealership disruption
- competitor zero-down-payment promotion
- interest-rate/pricing pressure following a repo-rate event

Pune is compared against **Nashik as a control** and the broader national context.

The dataset is synthetic but internally consistent and includes an evaluation-only ground-truth layer so the investigation pipeline can be tested objectively.

4. What the Prototype Demonstrates

The current prototype implements the core investigation loop.

Genuine anomaly detection

StratIQ identifies whether the KPI movement is meaningfully different from the target's historical behavior.

For the Suvidha scenario, the system detects the Pune decline and validates it against a control region.

Multi-source evidence fusion

StratIQ combines:

Structured evidence

- loan disbursals

- branch staffing
- dealership footfall
- credit-policy changes
- repo-rate events
- competitor events

Unstructured evidence

- dealer notes
- customer tickets
- local news
- social feedback

This allows numerical business signals and human/contextual evidence to be investigated together.

-> Competing hypotheses

Instead of immediately selecting one explanation, StratIQ generates multiple plausible hypotheses and ranks them.

-> Evidence-based confidence

Each hypothesis is evaluated using:

- **Evidence strength**
- **Temporal alignment**
- **Comparative analysis**

The final confidence score is computed by the analytical layer rather than simply being asserted by the LLM.

-> Contradictory evidence

Where available, the system preserves evidence that challenges a hypothesis rather than showing only confirming information.

-> Business recommendation

The final output translates the analysis into a decision-ready narrative:

Signal → Claim → Evidence → Confidence → Next Action

-> Ambiguity-aware architecture

If evidence is insufficient to confidently select one explanation, the intended system behavior is to abstain from false certainty and identify the evidence gap and next investigation step.

5. How the System Works

Layer 1 — Signal Engine

File: backend/signal_engine.py

The Signal Engine is the first gate.

It uses historical KPI behavior and change-point detection to determine whether a movement is unusual enough to investigate.

For the prototype:

Pune historical series

↓

Change-point detection

↓

Anomaly window

↓

Nashik control comparison

↓

Localized vs. broader movement

This prevents the reasoning layer from investigating every ordinary fluctuation.

Layer 2 — Evidence Fusion

Files:

- backend/evidence_fusion.py
- backend/vector_store.py

Once a meaningful anomaly is detected, StratIQ builds an evidence bundle around the anomaly window.

Structured evidence

Retrieved from SQLite using the relevant:

- entity
- time window
- business dimension

Unstructured evidence

Retrieved semantically from the indexed JSON evidence sources.

The result is a unified:

EvidenceBundle

containing the signal, structured evidence, unstructured evidence, and source metadata.

Layer 3 — Hypothesis Engine

Files:

- backend/hypothesis_engine.py
- backend/llm_client.py

The LLM receives the evidence bundle and generates **3–5 competing explanations**.

The model is instructed to ground hypotheses in the supplied evidence and reference the evidence items it uses.

Importantly, the LLM does **not** provide the final confidence score.

The analytical layer computes:

Evidence Strength

+

Temporal Alignment

+

Comparative Analysis

↓

Final Confidence

This separates **language-model reasoning** from **deterministic analytical scoring**.

Layer 4 — Ambiguity Layer

File: backend/ambiguity_layer.py

A business analyst should not manufacture certainty.

The Ambiguity Layer evaluates the ranked hypotheses against a confidence threshold.

When sufficient evidence exists:

Leading explanation

↓

Recommended action

When evidence is insufficient:

Competing explanations

↓

Evidence gaps

↓

Required next data/person

This is designed to prevent the system from turning uncertainty into a false definitive answer.

Layer 5 — Narrative

File: backend/narrative.py

The analytical result is converted into a business-facing format:

CLAIM

↓

EVIDENCE

↓

CONFIDENCE

↓

EVIDENCE GAP

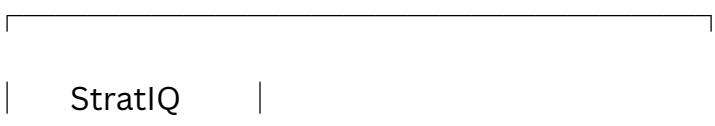
↓

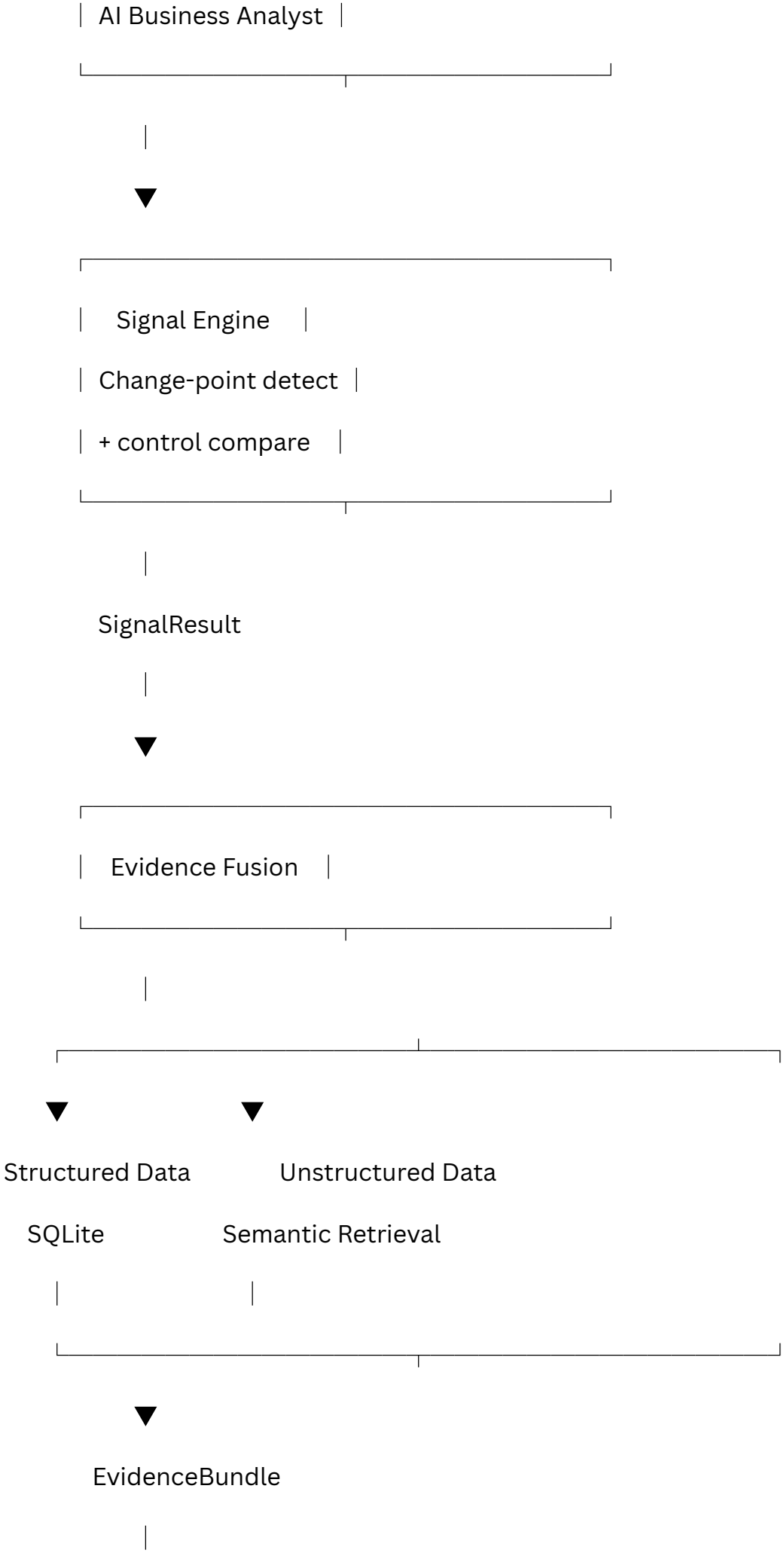
NEXT ACTION

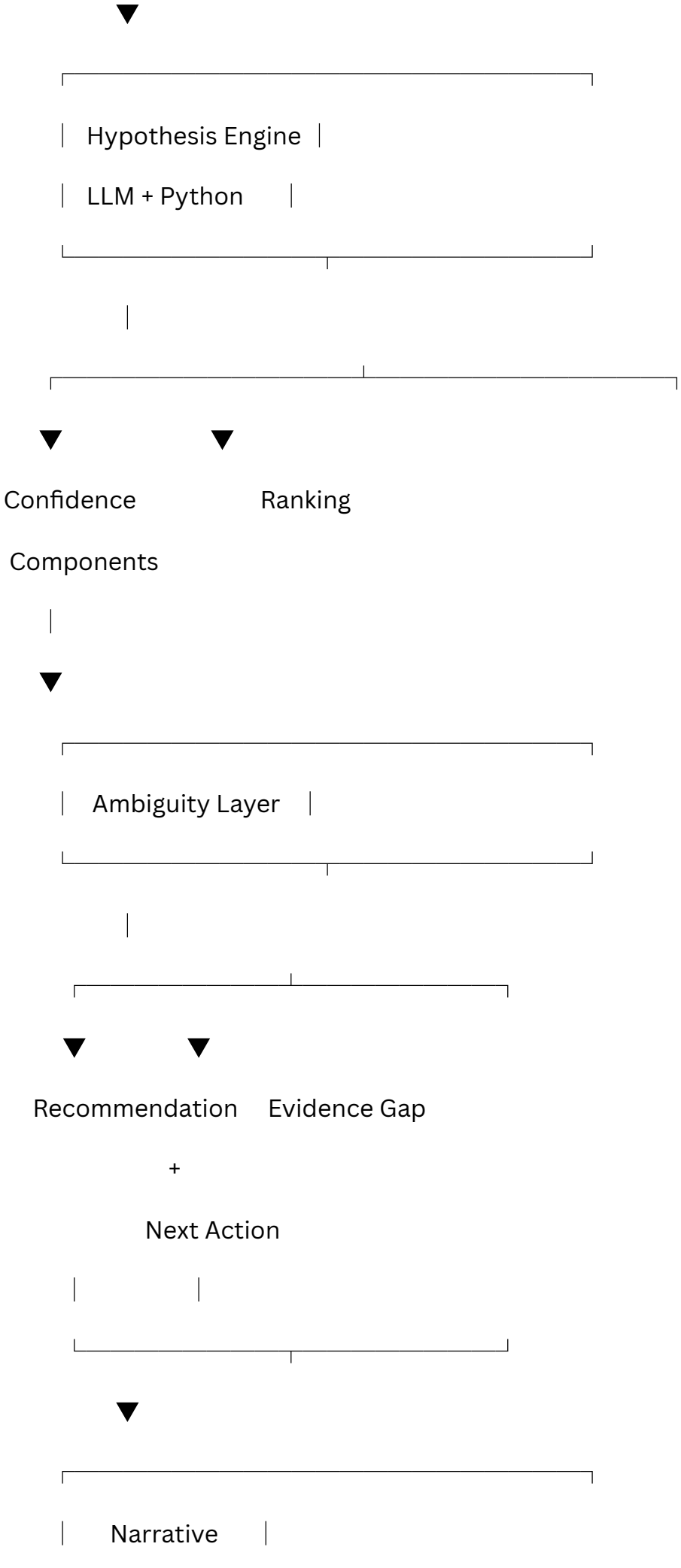
The objective is not to produce a longer AI explanation.

The objective is to produce a **decision-ready explanation**.

6. Architecture




AI Business Analyst
Signal Engine
Change-point detect
+ control compare
SignalResult
Evidence Fusion
Structured Data
SQLite
Unstructured Data
Semantic Retrieval
EvidenceBundle


Hypothesis Engine
LLM + Python
Confidence
Components
Ranking
Ambiguity Layer
Recommendation
Evidence Gap
+
Next Action
Narrative

7. LLM vs. Non-LLM Processing

A deliberate design decision in StratIQ is to **not make the LLM responsible for everything**.

Non-LLM processing

Python handles:

- data loading
- SQL retrieval
- statistical analysis
- change-point detection
- anomaly-window identification
- evidence aggregation
- temporal calculations
- comparative analysis
- confidence scoring
- ranking
- threshold logic

LLM processing

Gemini handles:

- interpretation of the evidence bundle
- generation of competing hypotheses
- explanation/mechanism drafting
- identification of supporting and contradictory evidence

This separation makes the system more transparent and reduces dependence on the LLM for deterministic analytical tasks.

8. Technology Stack

Layer	Technology
Language	Python
Backend API	FastAPI
Structured data	SQLite
Data analysis	pandas, NumPy
Signal detection	ruptures
Semantic retrieval	Current retrieval implementation in vector_store.py
LLM	Google Gemini via google-genai
Validation	Pydantic
Frontend	Streamlit
Testing	pytest
Configuration	python-dotenv
Deployment	Docker / Cloud Run configuration

9. Repository Structure

causalboard/

```
│
├── README.md
├── requirements.txt
├── Dockerfile
├── .env.example
│
```

```
├── data/
│   ├── scenario_config.py
│   ├── generate_synthetic_data.py
│   ├── suvidha.db
│   ├── unstructured/
│   └── ground_truth/
│
├── backend/
│   ├── models.py
│   ├── signal_engine.py
│   ├── vector_store.py
│   ├── evidence_fusion.py
│   ├── llm_client.py
│   ├── hypothesis_engine.py
│   ├── ambiguity_layer.py
│   ├── narrative.py
│   └── main.py
│
├── frontend/
│   └── app.py
│
├── tests/
│   └── test_pipeline.py
│
├── deploy/
│   └── deploy_cloudrun.sh
```

```
│
└──── docs/
    └──── demo_script.md
```

10. Setup

Requirements

Python 3.10+ is recommended.

Installation

```
git clone <YOUR-GITHUB-REPOSITORY>
cd causalboard


python -m venv venv
```

Windows

```
venv\Scripts\activate
```

macOS / Linux

```
source venv/bin/activate
```

Install dependencies:

```
pip install -r requirements.txt
```

11. Configure Gemini

Create .env in the project root:

```
GEMINI_API_KEY=your_actual_key
```

.env is intentionally excluded from Git.

For repository sharing, use:

.env.example

with:

```
GEMINI_API_KEY=your_key_here
```

12. Generate the Demo Dataset

The Suvidha scenario is generated reproducibly.

python data/generate_synthetic_data.py

This creates the structured database and unstructured evidence required for the investigation.

13. Run Tests

python -m pytest tests/ -q

14. Run the Backend

uvicorn backend.main:app --reload

The investigation endpoint is exposed through the FastAPI application.

15. Run the Interactive Prototype

streamlit run frontend/app.py

The Streamlit prototype allows the user to:

1. view the KPI movement,
2. investigate the anomaly,
3. see whether the signal is genuine,
4. inspect ranked hypotheses,
5. review evidence and scoring,
6. view the resulting business narrative and recommended action.

16. Example Prototype Flow

Step 1 — KPI movement

The user sees the Pune two-wheeler disbursal trend alongside Nashik control context.

Step 2 — Investigation

The user selects:

Investigate Pune anomaly

Step 3 — Signal confirmation

StratIQ identifies the anomaly window and compares Pune against the control region.

Step 4 — Evidence fusion

The engine retrieves both structured and unstructured evidence surrounding the anomaly.

Step 5 — Hypothesis generation

Gemini generates competing explanations grounded in the evidence bundle.

Step 6 — Confidence scoring

The analytical layer evaluates:

- evidence strength
- temporal alignment
- comparative analysis

and ranks the hypotheses.

Step 7 — Decision output

The interface presents the leading explanation, evidence, confidence, and recommended next action.

17. Validation Results — Suvidha Prototype

The prototype has been validated against the synthetic Suvidha scenario.

Signal detection

Pune disbursals exhibit the engineered decline, while the control region does not show a comparable movement.

Evidence fusion

The evidence layer retrieves both:

- structured business records
- relevant dealer/customer/news/social evidence

across the anomaly window.

Hypothesis generation

The engine produces multiple competing explanations rather than a single unqualified answer.

Confidence scoring

The final ranking is calculated from explicit analytical components rather than LLM-generated confidence assertions.

Ground truth

The synthetic scenario contains an evaluation-only ground-truth layer that is **not exposed to the reasoning pipeline**.

This allows the prototype to be evaluated without giving the model the answer.

18. Deliberate Design Choices

Evidence before explanation

The system retrieves evidence before asking the LLM to explain the anomaly.

LLM for reasoning, Python for measurement

The LLM generates candidate explanations, while deterministic logic calculates analytical scores.

Competing hypotheses instead of one narrative

The system explicitly maintains multiple possible explanations.

Uncertainty is a first-class output

The architecture allows the system to abstain when available evidence cannot reliably distinguish between competing explanations.

19. Current Prototype Scope

This submission intentionally focuses on the core investigation loop:

Signal

→ Evidence

→ Hypotheses

→ Confidence

→ Action / Ambiguity

The current prototype does **not** attempt to represent the full enterprise product.

Features such as multi-persona narratives, role-based entitlements, sparse-history handling, richer KPI semantic contracts, runtime telemetry, and production-grade data connectors are part of the next development layer rather than being artificially simulated in this prototype.

This keeps the submitted system focused on demonstrating the central problem-solving capability.

20. Beyond the Prototype

Suvidha is the **proving ground, not the product boundary**.

The long-term goal is a business-agnostic StratIQ engine that can investigate anomalies across industries and data structures.

Universal Data Adapter

Accept:

- CSV
- Excel
- database tables
- unstructured documents

and automatically infer:

- time
- KPI
- entities
- dimensions
- relationships

so the same analytical engines can operate without rewriting the core pipeline.

Adaptive Reasoning

Future versions can retrieve and apply different business-analysis frameworks depending on the anomaly type—for example:

- operational/root-cause analysis
- market analysis
- issue-tree decomposition
- competitive analysis

Enterprise Integration

Future connectors can extend StratIQ to:

- ERP
- CRM
- finance
- operations
- customer support
- external intelligence

The architecture is designed so that **the same reasoning loop can travel across businesses.**

21. Vision

Most business intelligence answers:

What happened?

StratIQ is built to continue the investigation:

Was it real?
Why did it happen?
How strong is the evidence?
What should we do next?

StratIQ

From what changed, to why, to what next.